ENGINEERING CAPSTONE PROJECT – 1

ENTERPRISE ROLE-BASED ACCESS CONTROL (RBAC) & IDENTITY AUDITOR

Full Project Documentation / Research-Style Report

Team 16
2320030047 – P. Varun Reddy
2320030410 – K. Chandana
2320030446 – Pardhava Nandan

Under the Guidance of
Dr. K. Swanthana
Assistant Professor
Department of Computer Science and Engineering

KONERU LAKSHMAIAH EDUCATION FOUNDATION
Andhra Pradesh, India

Abstract

Enterprise applications need to ensure that users receive only the permissions required for their work. RBAC assigns permissions to roles and roles to users, but outdated or unnecessary permissions may remain when responsibilities change. This privilege creep can increase security exposure. The Enterprise RBAC and Identity Auditor combines user and role administration, permission-based authorization, audit logging, privilege review, and rule-based risk classification. The documented stack includes Java, Spring Boot, Spring Security, JWT, BCrypt, MySQL, Spring Data JPA/Hibernate, and React.js. Available materials describe functional views and testing areas but do not provide measured accuracy, latency, or scalability results; quantitative evaluation remains future work.

Keywords: RBAC, identity auditing, privilege creep, least privilege, risk assessment, audit logging.

1. Introduction

Enterprise applications serve users with different responsibilities. RBAC organizes access into roles, but roles and permissions need ongoing maintenance. Old access may remain after a role change, producing privilege creep. The project combines authorization and identity auditing to improve visibility into accounts, permissions, events, and risk indicators.

1.1 Problem Statement

A system needs to manage user-role-permission relationships, enforce permission checks, record relevant security events, and flag potentially excessive or stale access for review.

1.2 Objectives

• Manage users, roles, permissions and mappings.

• Authenticate users and protect endpoints.

• Enforce ALLOW/DENY decisions.

• Record security-related events.

• Identify potentially excessive or unused access.

• Display explainable risk scores and categories.

2. Background and Related Work

RBAC research formalizes role and permission relationships. Role-mining research examines role derivation from user-permission assignments. Risk-based access-control research studies contextual indicators that can support access decisions or review priority. Security guidance emphasizes least privilege, account management, and auditing.

Source	Focus	Relevance
Sandhu et al. (1996)	RBAC models	Role-permission structure.
Ferraiolo et al. (2001)	NIST RBAC proposal	Authorization concepts.
Atlam et al. (2020)	Risk-based access control review	Risk factors and estimation.
Parkinson & Khana (2022)	High-risk over-entitlement	Excessive permission review.
NIST SP 800-53 Rev. 5	Security controls	Access control and audit practices.

2.1 Research Gap

The qualitative gap is the separation between granting permissions and reviewing whether they remain appropriate. The project combines RBAC administration, audit logging, privilege review, and rule-based risk classification. No measured percentage for this gap is supplied.

2.2 Research Questions

• How are role-permission mappings used to enforce ALLOW/DENY decisions?

• How can audit records support identity reviews?

• Can a transparent score help prioritize review?

• Which metrics are needed to validate the system?

3. System Requirements and Scope

3.1 Actors

Actor	Responsibility
Standard user	Logs in and requests permitted functions.
Administrator	Manages users, roles, permissions, and reviews.
RBAC service	Checks required permissions.
Audit service	Records security events.
Risk service	Evaluates configured indicators.
Database	Stores identities and audit information.

3.2 Functional Requirements

• Secure registration/login.

• User and role management.

• Role-permission configuration.

• Protected API authorization.

• Audit event recording.

• Privilege and identity-risk review.

• Risk category/reason display.

3.3 Non-Functional Requirements

• Security and password hashing.

• Auditability and protected records.

• Maintainable separation of components.

• Clear interface and risk explanations.

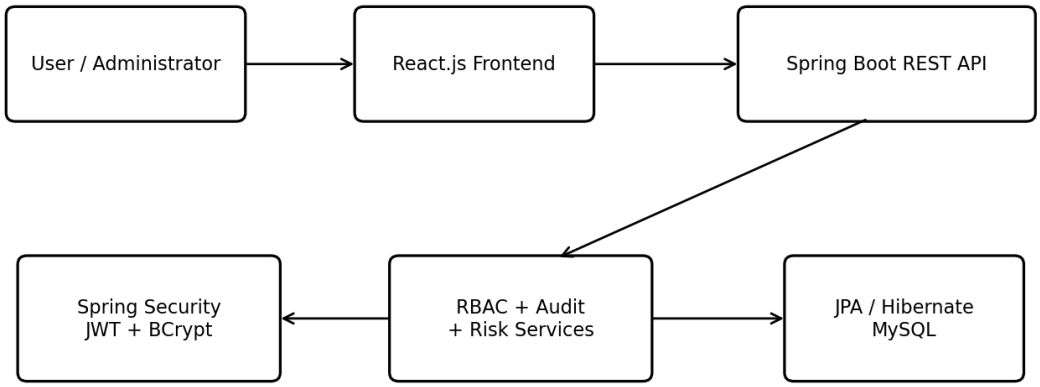
• Scalability to be measured, not assumed.

3.4 Scope Boundaries

This report follows the more recent stack described in the research draft and Review-2. Automated recommendations and quantitative anomaly detection should be described as implemented only if verified in the final source code.

4. Proposed System Architecture

The system is described as a layered web application: React.js frontend, Spring Boot REST services, security integration, RBAC/audit/risk services, and MySQL persistence.



Conceptual architecture based on the documented project stack

Figure 1. Conceptual architecture based on project materials.

Layer	Technology	Purpose
Frontend	React.js	User interface.
API	Java/Spring Boot	Backend services.
Security	Spring Security, JWT, BCrypt	Endpoint protection and password handling.
Services	RBAC, audit, risk	Authorization and review.
Persistence	JPA/Hibernate, MySQL	Data storage.

4.1 Frontend–Backend Communication

The frontend calls backend REST APIs. The backend validates the request, applies security and permission rules, performs the permitted operation, and returns a response. Exact endpoint names should be copied from the actual code.

5. RBAC Model and Data Relationships

Users receive permissions through assigned roles. A user-role mapping connects users and roles; a role-permission mapping connects roles and permitted operations.

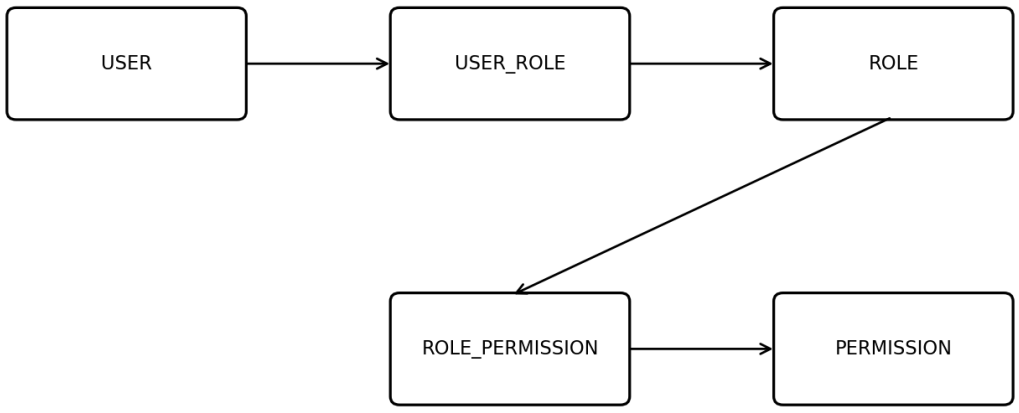


Figure 2. Logical relationship illustration; verify entity names against source code.

Entity	Representative information
User	Identity, status, password hash.
Role	Name and description.
Permission	Permission token/action.
User–Role	User and role mapping.
Role–Permission	Role and permission mapping.
Audit event	Actor, action, timestamp, decision.
Risk assessment	Score, category, reasons.

5.1 Example Permission Tokens

Token	Illustrative meaning
USER_READ	View users.
USER_CREATE	Create account.
ROLE_MANAGE	Manage roles/permissions.
AUDIT_READ	View audit events.
RISK_READ	View risk assessments.
REPORT_VIEW / REPORT_EXPORT	View/export reports.

6. Authentication and Authorization Workflow

Authentication verifies identity. Authorization determines whether the authenticated user may perform the requested operation. Successful login does not grant unrestricted access.

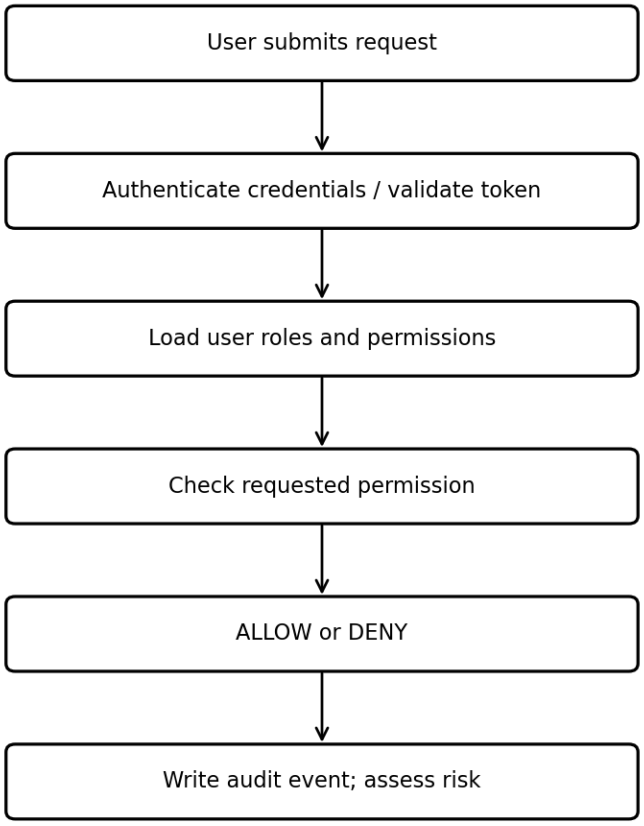


Figure 3. Request-processing workflow.

6.1 Workflow

1. Submit credentials or request token.
2. Validate identity/token.
3. Load assigned roles and permissions.
4. Check the requested permission.
5. Allow or deny the operation.
6. Record relevant audit event and assess review indicators.

6.2 Security Components

Component	Purpose
BCrypt	Password hashing; no plaintext storage.
JWT	Token-based request/session handling.
Spring Security	Protect endpoints.
RBAC check	Enforce permission rules independently of risk score.

7. Identity Auditing and Privilege Creep

Privilege creep is the accumulation of permissions that a user no longer needs as responsibilities change. Identity auditing reviews whether current access is still appropriate.

7.1 Conditions for Review

• Unjustified elevated access.

• Permissions beyond expected role.

• Unused privileges for 30+ days as an illustrative rule.

• Unknown device/IP context.

• Unusual login times or suspicious sessions.

• Repeated denied requests.

• Potentially high-risk role assignment.

7.2 Review Process

1. Identify the flagged account/permission.

2. Inspect the reason and current responsibilities.

3. Check with the owner/manager if needed.

4. Retain, adjust, or remove access based on policy.

5. Record the decision and justification.

Unused access should not be automatically removed without checking business need.

8. Risk Scoring Method

Configured rule weights contribute to a review-priority score. These are heuristic values, not calibrated probabilities.

Risk factor	Weight
Elevated privilege	+25
Excessive permissions	+25
Unused privilege 30+ days	+20
Unknown IP	+5
Unknown device	+20
Login 11 p.m.–6 a.m.	+15
Failed attempt	+5
Suspicious session	+25
Repeated unauthorized access	+30
High-risk role	+25

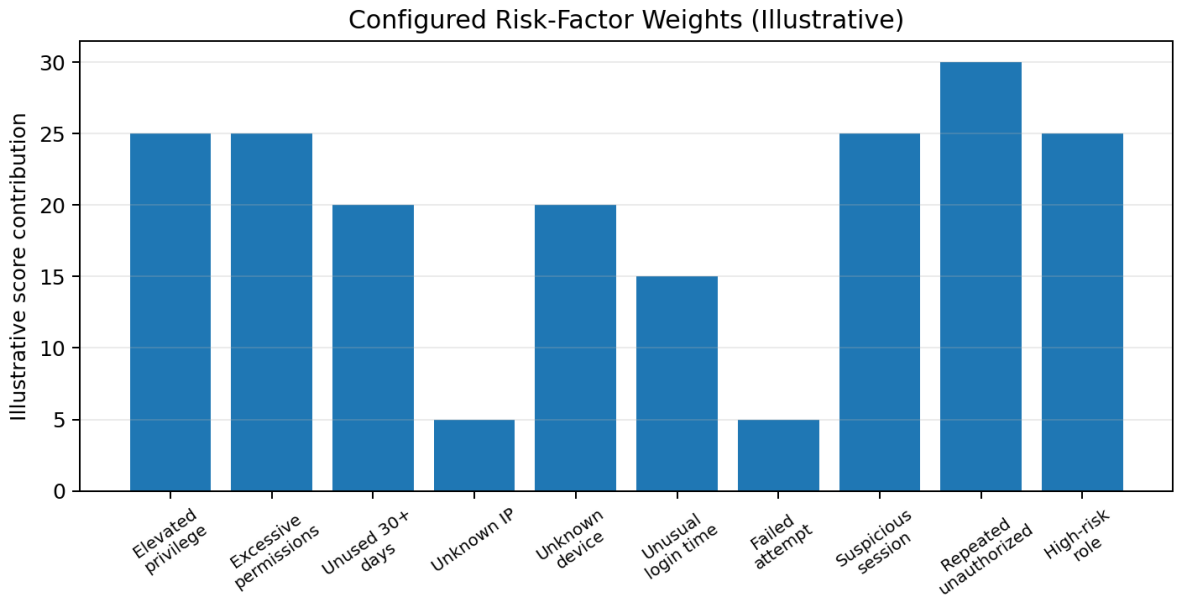


Figure 4. Illustrative configured risk-factor weights.

8.1 Score Bands

An explanatory additive model is S = sum(w■ × x■), where x■ indicates whether a condition applies. Confirm score cap, normalization, and repeated-event handling in code.

Category	Range
Low	0–20
Medium	21–40
High	41–60
Critical	61+

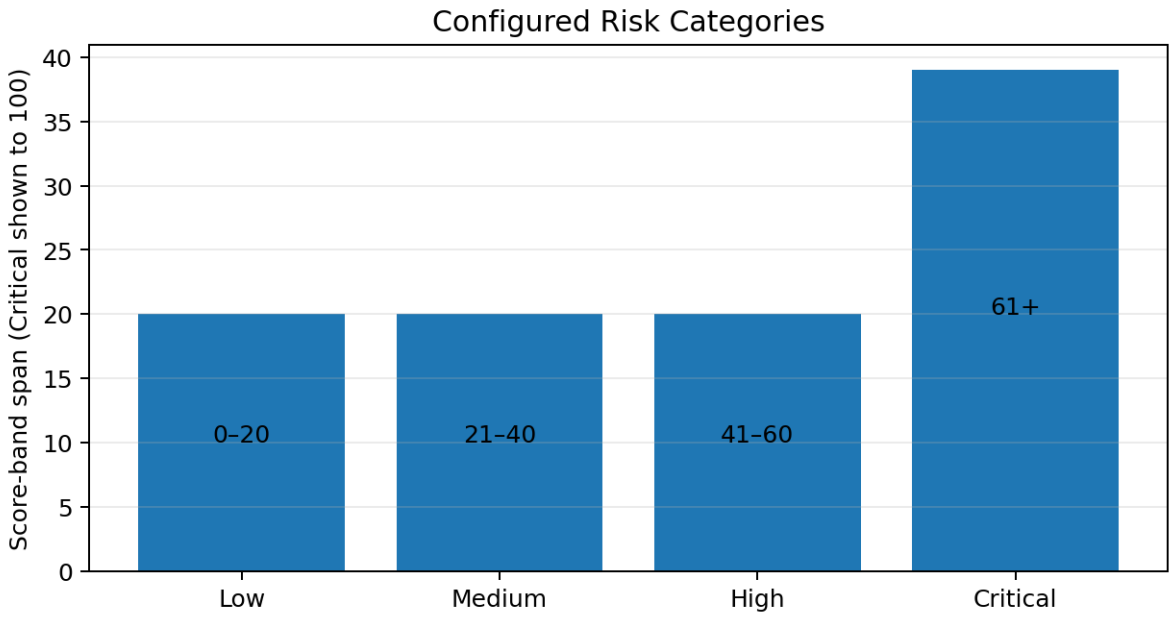


Figure 5. Configured risk categories.

9. Worked Risk Scenarios

Scenario	Factors	Score/category
A	Unknown device (+20)	20 / Low
B	Unknown device + unusual time	35 / Medium
C	Elevated + excessive permissions	50 / High
D	Elevated + excessive + unused	70 / Critical

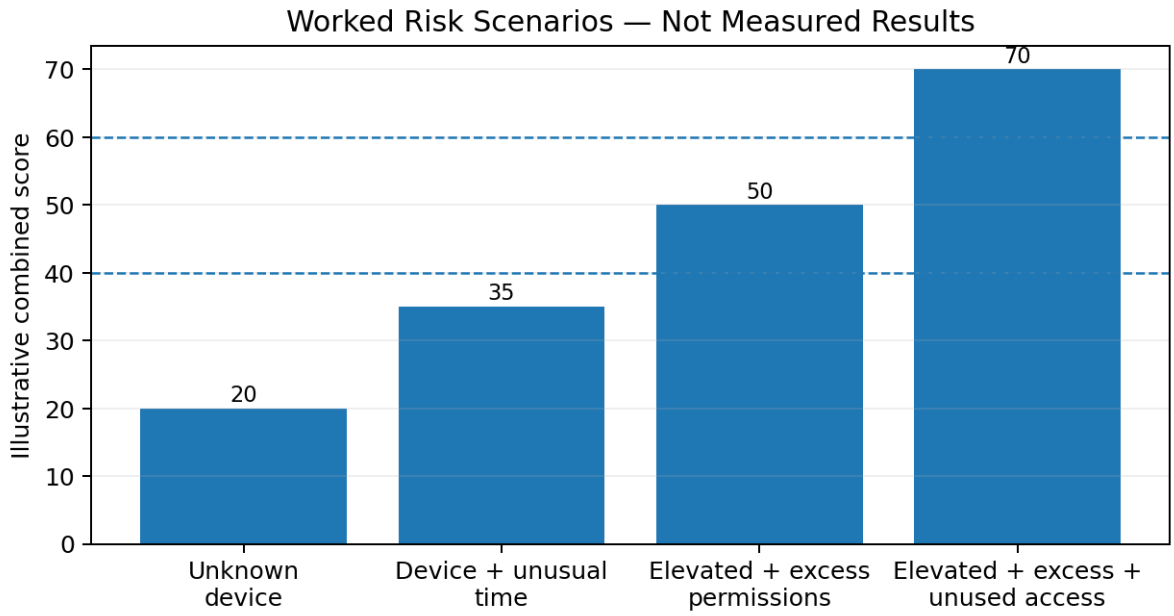


Figure 6. Illustrative calculations, not measured application results.

9.1 Interpretation

Risk categories prioritize review; they do not prove malicious intent. Missing required permissions must result in DENY regardless of score. A high score should trigger policy-based review.

9.2 Items to Confirm

• Is each factor counted once or repeatedly?

• Is there a score cap?

• What threshold defines repeated unauthorized access?

• How are suspicious sessions identified?

10. Audit Logging and Traceability

Audit logs support accountability and investigation by recording who performed an action, what happened, which resource was affected, when it occurred, and the decision.

Field	Purpose
Event ID	Unique reference.
Actor	Account performing action.
Action/resource	Operation and target.
Timestamp	Chronology.
Decision	ALLOW/DENY/outcome.
Context	Relevant IP/device/session, subject to privacy.
Review note	Administrative decision.

10.1 Audit-Log Protection

• Restrict access to authorized users.

• Do not log passwords or tokens.

• Protect records from unauthorized changes.

• Set retention rules.

• Test persistence after restart.

10.2 Audit Workflow

Capture event → associate with identity → inspect permissions/context → evaluate configured indicators → record administrator decision.

11. Implementation Environment and Stack

Technology	Role
Java	Backend language.
Spring Boot	Backend/REST services.
Spring Security	Endpoint security.
JWT	Token handling.
BCrypt	Password hashing.
MySQL	Relational database.
Spring Data JPA/Hibernate	Persistence.
React.js	Frontend.
Spring AOP	Cross-cutting behavior where configured.
Maven	Build/dependencies.
Postman	API testing.
Swagger/OpenAPI	API documentation.
Windows 10/11	Development environment listed in materials.

11.1 Layered Implementation

• Controllers expose API endpoints.

• Services implement business logic.

• Repositories communicate with MySQL.

• React calls REST APIs and renders responses.

• Security configuration protects endpoints.

Earlier Review-1 materials mention an alternative stack. Verify the final source code and use one consistent stack in the submitted report.

12. Frontend Interface and User Experience

The project draft describes login, identity management, role/authorization policy, permission registry, and privilege-creep/risk views. Add screenshots from the actual running application where required.

12.1 Login

Provides credential-based entry and should avoid exposing sensitive internal details in error messages.

12.2 Identity Management

The described screen displays account names, emails, roles, account status, risk scores, and administrative actions.

12.3 Role and Permission Views

Role management shows role definitions and associated permissions. The registry lists granular tokens such as USER_READ, ROLE_MANAGE, AUDIT_READ, RISK_READ, REPORT_VIEW, and REPORT_EXPORT.

12.4 Risk View

The draft shows sample account classifications, including a Medium-risk administrator score of 45 and two Low-risk accounts with score 0. These are demonstration values, not benchmark results.

13. Testing Methodology

Testing should cover functional behavior, security, audit capture, risk-rule correctness, and operational characteristics. The available materials do not provide a complete execution log; the following are planned cases.

Test	Action	Expected outcome
Login	Valid/invalid credentials	Accept valid; reject invalid.
Password	Inspect stored credential	BCrypt hash.
Role assignment	Assign role	Mapping persists.
Permission	Update role permissions	Updated mapping used.
Authorization	Try with/without permission	Allow/deny correctly.
Audit	Perform security action	Event recorded.
Risk	Seed known factors	Expected score band.
Persistence	Restart and inspect records	Records persist as intended.

13.1 Test Data

Use employee, analyst, and administrator test roles. Include benign cases and seeded issues such as excessive permissions, stale access, unknown device context, unusual login time, failed attempts, and repeated denied access.

14. Evaluation Metrics and Evidence Status

Metric	Definition	Status
Precision	True positives / all flags	Not supplied
Recall	Detected seeded issues / all seeded issues	Not supplied
F1-score	Harmonic mean of precision and recall	Not supplied
False-positive rate	Benign cases incorrectly flagged	Not supplied
Detection time	Elapsed time to flag issue	Not supplied
Audit coverage	Expected events recorded	Needs test evidence
Scalability	Latency/resource change with load	Not measured

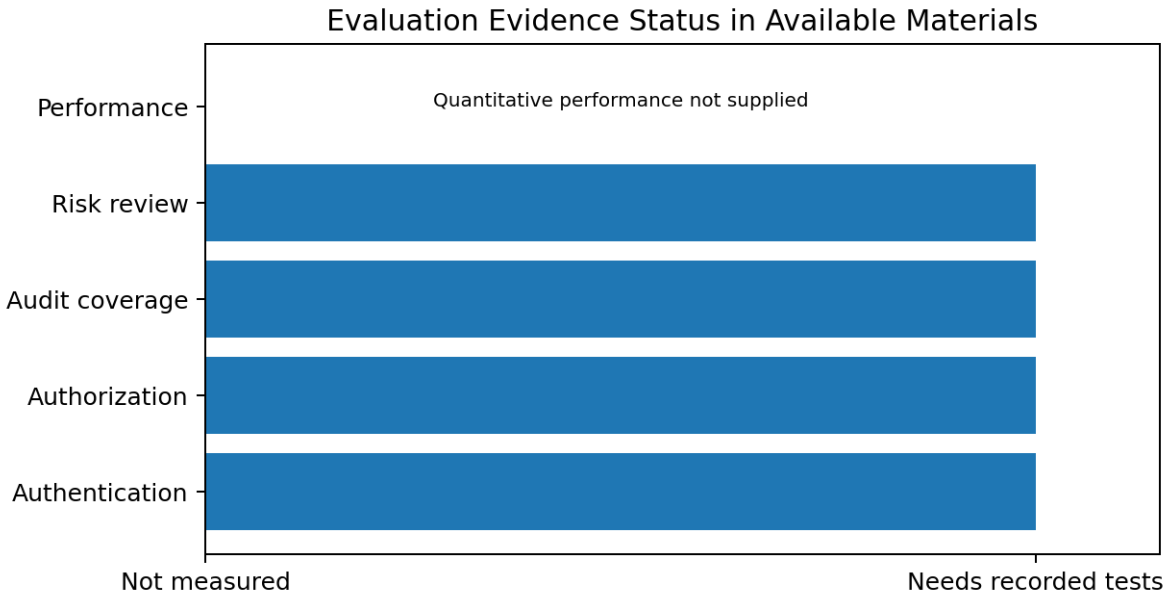


Figure 7. Quantitative results are not supplied in current materials.

14.1 Reproducibility

• Record source version and environment.

• Document users, roles, and permissions.

• Save expected and observed outcomes.

• Record score components and audit events.

• Repeat tests after restarts.

• Publish metrics only after calculating from real tests.

15. Results and Discussion

Available materials describe registration/login, account management, role/permission mapping, authorization, privilege review, audit logging, and risk classification. The research draft includes screenshots of login, identity management, role policy, permission registry, and risk review.

This supports a functional description but does not establish a statistically validated detection rate or performance benchmark. Example scores illustrate configured rules and must not be presented as experimental outcomes.

15.1 Research Gap Discussion

The intended benefit is visibility: administrators can inspect assigned permissions, review event history, and see which configured factors contributed to a score. Whether this reduces unnecessary permissions must be tested against a defined baseline.

15.2 Comparison

Capability	Basic RBAC	Project workflow
Role-based authorization	Yes	Yes
Identity management	Common	Described
Audit visibility	Implementation-dependent	Described
Privilege review	Additional process	Risk-review view described
Quantitative improvement	Must be measured	Not established

16. Limitations, Ethics, and Security

16.1 Technical Limitations

• Weights and thresholds are heuristic.

• Aggregation and repeated-event handling need confirmation.

• Risk-factor definitions need precise rules.

• No measured precision, recall, latency, or scalability is supplied.

• Sample scores are not benchmark evidence.

16.2 Operational Considerations

• Review flags with business context.

• Do not remove permissions without checking need.

• Protect logs and sensitive information.

• Document exceptions and decisions.

• Restrict administrative actions.

16.3 Responsible Use

Use the system for authorized security administration. Risk scores should support review and must not be treated as definitive judgments about a person or account.

17. Future Scope and Project Plan

Phase	Activity	Deliverable
1	Verify code and report claims	Feature inventory
2	Build labeled test data	Dataset
3	Run functional tests	Test log
4	Validate risk rules	Rule specification
5	Measure performance	Metric tables/charts
6	Improve review workflow	Recommendations
7	Finalize documentation	Evidence-backed report

17.1 Future Enhancements

• Scheduled or event-driven reviews.

• Auditable permission recommendations.

• Improved reports and alerts.

• Validated weights and thresholds.

• Large-scale and usability testing.

• Stronger audit integrity and retention.

18. Conclusion

The Enterprise RBAC and Identity Auditor is designed to combine user/role administration, permission-based authorization, secure authentication, audit logging, privilege-creep awareness, and explainable risk classification. The documented stack includes Java, Spring Boot, Spring Security, JWT, BCrypt, MySQL, JPA/Hibernate, and React.js.

Its principal contribution is integrating authorization and identity review. The available interface descriptions show how permissions, audit activity, and risk indicators can be reviewed together. Risk weights remain heuristic, and quantitative detection/performance results have not been supplied. Future work should verify the implementation and run reproducible tests.

Acknowledgment

The team acknowledges the guidance of Dr. K. Swanthana and the academic support of Koneru Lakshmaiah Education Foundation.

Author Contributions

List only actual contributions. Assign frontend, backend, database, security, testing, research, documentation, and presentation tasks to the team members who performed them.

References

[1] R. S. Sandhu et al., “Role-Based Access Control Models,” IEEE Computer, 29(2), 38–47, 1996.

[2] D. F. Ferraiolo, D. R. Kuhn, and R. Chandramouli, “Proposed NIST Standard for Role-Based Access Control,” ACM TISSEC, 4(3), 224–274, 2001.

[3] H. F. Atlam et al., “Risk-Based Access Control Model: A Systematic Literature Review,” Future Internet, 12(6), 103, 2020, doi:10.3390/fi12060103.

[4] S. Parkinson and S. Khana, “Identifying high-risk over-entitlement in access control policies using fuzzy logic,” Cybersecurity, 5, 6, 2022, doi:10.1186/s42400-022-00112-1.

[5] NIST, Security and Privacy Controls for Information Systems and Organizations, SP 800-53 Rev. 5, 2020.

[6] J. H. Saltzer and M. D. Schroeder, “The Protection of Information in Computer Systems,” Proceedings of the IEEE, 63(9), 1278–1308, 1975.

[7] Team 16, Enterprise RBAC & Identity Auditor project review and research draft, KLEF, 2026.

Journal Homepage / Related Article

Article: Risk-Based Access Control Model: A Systematic Literature Review.

Journal: Future Internet, vol. 12, issue 6, article 103 (2020).

Journal homepage: https://www.mdpi.com/journal/futureinternet

Article page: https://www.mdpi.com/1999-5903/12/6/103

DOI: https://doi.org/10.3390/fi12060103

This is a related research article, not a publication by the student team. Do not imply journal acceptance or use journal branding to represent this project as published.

Appendix A. Figure Index

Figure	Description
1	Conceptual architecture
2	User-role-permission relationships
3	Authentication/authorization flow
4	Risk-factor weights
5	Risk score categories
6	Worked risk scenarios
7	Evaluation evidence status

Appendix B. Glossary

Term	Meaning
RBAC	Role-Based Access Control.
Privilege creep	Accumulation of unnecessary permissions.
Least privilege	Minimum access needed for duties.
Authentication	Verify identity.
Authorization	Check permission.
JWT	JSON Web Token.
BCrypt	Password-hashing function.
Audit event	Record of a security-related action.
Risk score	Review-priority indicator, not a probability by itself.

Appendix C. Submission Checklist

• Confirm team names, IDs, guide, and department.

• Verify technology stack against final source code.

• Insert genuine application screenshots.

• Confirm score formula and thresholds in code.

• Run tests and record actual results.

• Do not invent dataset sizes or performance metrics.

• Verify DOI/reference details.

• Ensure personal contribution statements are accurate.

Appendix D. Sample Test Record

Test ID	Setup/input	Expected	Observed	Pass/fail
T-01	Valid login			
T-02	Invalid login			
T-03	Endpoint with permission			
T-04	Endpoint without permission			
T-05	Excessive permission case			
T-06	Audit event generated			
T-07	Known risk factors			
T-08	Restart/persistence			